Jenkins — Complete Guide

From Beginner to Pro: CI/CD Concepts, Pipelines, and Real Practice

DevOps Learning Notes

July 2026

Contents

How to Use This Guide

This guide takes you from **“what is CI/CD”** through **installing Jenkins, building jobs and pipelines, integrating with Git/Docker/Kubernetes, securing and scaling Jenkins**, and finishes with a **full real-world pipeline project**, a **command/syntax cheat sheet**, and **interview prep**. Every topic explains *what it is*, *why it exists*, *when/where to use it*, and includes a runnable example.

Part 1 — CI/CD & Jenkins Fundamentals

1.1 What is CI/CD?

- **CI (Continuous Integration)** — every time a developer pushes code, it is automatically built and tested, so bugs are caught within minutes instead of weeks.
- **CD (Continuous Delivery/Deployment)** — after passing tests, the code is automatically packaged and pushed toward staging or production, either with a manual approval gate (**Delivery**) or fully automatically (**Deployment**).

Why it matters: before CI/CD, teams integrated code rarely (weekly/monthly), leading to painful "merge hell" and bugs discovered late. CI/CD makes integration a non-event — small changes, tested constantly, deployed confidently.

Here is the typical flow an automation server like Jenkins drives:

Figure 1: CI/CD Pipeline Flow

1.2 What is Jenkins?

Jenkins is a free, open-source **automation server** written in Java. It is the most widely used CI/CD tool in the industry, with 1800+ plugins connecting it to almost any tool (Git, Docker, Kubernetes, AWS, Slack, SonarQube, and more).

Why Jenkins specifically: - **Free and open-source**, self-hosted (full control over your infrastructure and data). - **Massive plugin ecosystem** — very few CI needs aren't covered by an existing plugin. - **Highly extensible** — pipelines are code (Groovy-based DSL), so builds can be as simple or as complex as needed. - **Distributed builds** — one controller can dispatch work across many agent machines (Linux, Windows, macOS, Docker containers). - **Mature and battle-tested** — created in 2011 (as a fork of Hudson), used by huge numbers of organizations worldwide.

1.3 Where Jenkins Fits in the DevOps Toolchain

```
Code (Git) -> Build (Jenkins + Maven/Gradle/npm) -> Test (Jenkins + JUnit/pytest)
  -> Quality Gate (SonarQube) -> Package (Docker) -> Registry (Docker Hub/ECR)
  -> Deploy (Jenkins + Ansible/Terraform/kubectl) -> Monitor (Prometheus/Grafana)
```

Jenkins is the **orchestrator** — it doesn't replace Git, Docker, or Kubernetes; it *calls* them in the right order, at the right time, and reports the result.

1.4 Jenkins vs Other CI/CD Tools

Tool	Style	Notes
Jenkins	Self-hosted, plugin-driven	Most flexible/customizable, but you manage the infrastructure
GitHub Actions	Hosted, YAML-based	Tightly integrated with GitHub repos, easy to start

Tool	Style	Notes
GitLab CI/CD	Hosted or self-hosted, YAML-based	Built into GitLab, strong for GitLab-centric teams
CircleCI	Hosted (SaaS)	Fast setup, good for cloud-native teams
Azure DevOps / TeamCity / Bamboo	Hosted/self-hosted	Enterprise-focused alternatives

Why choose Jenkins: when you need maximum control, on-prem/self-hosted infrastructure, complex custom workflows, or you're integrating with legacy/enterprise systems that hosted CI tools don't reach easily.

Part 2 — Installation & Architecture

2.1 Jenkins Architecture — Controller & Agents

Jenkins uses a **controller-agent (master-agent)** model:

- **Controller (Master)** — runs the web UI, stores job configurations, schedules builds, and manages plugins. In small setups it can also run builds itself, but that's discouraged in production.
- **Agent (Slave/Node)** — a separate machine (or container) that actually executes build steps, reporting results back to the controller. You can have agents running different OSes, tool versions, or hardware (e.g., a GPU agent for ML builds).

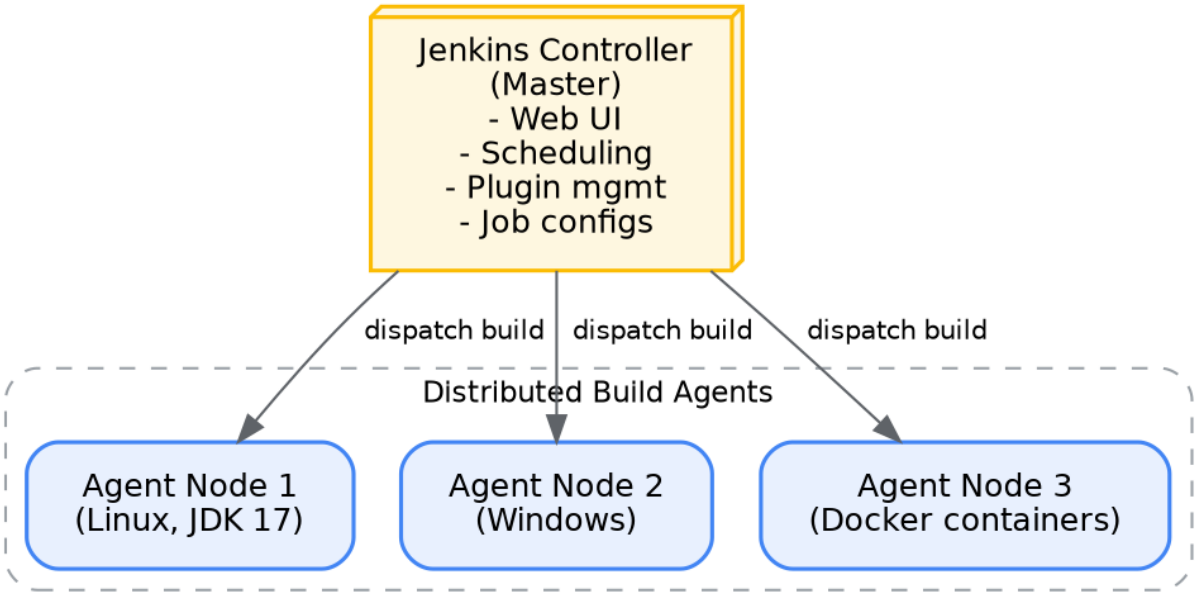

Figure 2: Jenkins Controller-Agent Architecture

Why separate controller and agents: isolates the controller from resource-heavy/untrusted build workloads, lets you scale horizontally (add more agents under load), and lets different jobs run on the right environment (Windows job on a Windows agent, ARM build on an ARM agent, etc.).

2.2 Installing Jenkins

Option A — Docker (fastest way to try Jenkins)

```
docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

Then visit `http://localhost:8080` — Jenkins will show an "unlock" screen asking for an initial admin password:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Option B — Native install (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y fontconfig openjdk-17-jre

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | \
  sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins

# Get the initial admin password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Option C — Kubernetes (Helm chart, production-grade)

```
helm repo add jenkins https://charts.jenkins.io
helm repo update
helm install jenkins jenkins/jenkins --namespace jenkins --create-namespace
```

2.3 First-Time Setup Wizard

1. Paste the initial admin password.
2. Choose **“Install suggested plugins”** (Git, Pipeline, Credentials Binding, etc. — good default set for beginners).
3. Create your first admin user.
4. Confirm the Jenkins URL (used for webhook callbacks, notifications).

2.4 Adding a Build Agent

Manage Jenkins → Nodes → New Node, then choose a launch method:

- **SSH** — controller connects to the agent machine over SSH and starts the agent JAR automatically. Best for persistent Linux/Windows machines.
- **Agent via Java Web Start (JNLP)** — the agent connects *out* to the controller — useful when the agent is behind a firewall/NAT the controller can’t reach directly.
- **Docker/Kubernetes** — agents are spun up dynamically as containers/pods per build and destroyed afterward (very common in modern setups — no idle capacity, always a clean environment).

```
// Kubernetes plugin — dynamic pod agent example (declared in a pipeline)
pipeline {
  agent {
    kubernetes {
      yaml """
        apiVersion: v1
        kind: Pod
        spec:
          containers:
```

```
          - name: maven
            image: maven:3.9-eclipse-temurin-17
            command: ['cat']
            tty: true
      """
    }
  }
  stages {
    stage('Build') {
      steps {
        container('maven') {
          sh 'mvn -B clean package'
        }
      }
    }
  }
}
```

2.5 Jenkins Home Directory — What's Inside

```
$JENKINS_HOME/
  jobs/                 <- every job's config + build history
  plugins/               <- installed plugins
  secrets/               <- credentials, master key
  config.xml             <- global Jenkins configuration
  nodes/                 <- agent definitions
  workspace/              <- default working directories for builds (per job)
```

Backup tip: in production, back up `$JENKINS_HOME` regularly (or use the ThinBackup / periodic backup plugins) — it contains every job's history and configuration.

Part 3 — Freestyle Jobs (The Classic Starting Point)

3.1 What is a Freestyle Job?

A **Freestyle job** is Jenkins' original, UI-configured job type — you fill in form fields (source repo, build steps, post-build actions) instead of writing code. Good for learning the concepts and very simple tasks, but doesn't scale well for complex logic and isn't stored as versioned code by default.

3.2 Creating a Freestyle Job (step by step)

1. **New Item → Freestyle project → name it → OK**
2. **Source Code Management** → choose Git → paste repository URL → specify branch (`*/main`).
3. **Build Triggers** → choose how it starts:
   - **Poll SCM** — Jenkins periodically checks Git for changes (`H/5 * * * *` = every ~5 minutes).
   - **GitHub hook trigger for GITScm polling** — Git notifies Jenkins instantly via web-hook (preferred — no polling delay).
   - **Build periodically** — cron-style schedule, regardless of code changes (e.g., nightly builds).
4. **Build Steps** → add "Execute shell" (Linux) or "Execute Windows batch command":

```
mvn clean package
```

5. **Post-build Actions** → e.g., "Archive the artifacts" (`target/*.jar`), "Publish JUnit test result report", or "Email notification".
6. **Save → Build Now**.

3.3 Why Freestyle Jobs Fall Short at Scale

- Configuration lives only in Jenkins' UI/XML — no code review, no version history tied to your app repo.
- Hard to express conditional logic, parallel stages, or complex multi-step workflows.
- Re-creating the same job for 10 microservices means clicking through the UI 10 times.

This is exactly why Pipelines (next section) became the standard — a pipeline is defined as code (`Jenkinsfile`), lives in your Git repo next to the app, and is reviewed/versioned like any other code.

Part 4 — Jenkins Pipelines

4.1 What is a Pipeline?

A **Pipeline** is a suite of plugins that lets you define your entire build/test/deploy process as **code**, written in Groovy-based DSL, usually stored in a file called `Jenkinsfile` at the root of your repository.

Why Pipelines over Freestyle jobs: - **Pipeline as Code** — versioned, reviewed, and branched right alongside your application code. - **Resilience** — pipelines survive a Jenkins controller restart mid-build (with `checkpoint`/durable task support). - **Complex workflows** — parallel stages, conditional logic, input/approval steps, retries. - **Visualization** — the Stage View / Blue Ocean UI shows exactly which stage is running or failed.

4.2 Declarative vs Scripted Pipelines

	Declarative	Scripted
Syntax	Structured, opinionated (`pipeline { stages { ... } }`)	Full Groovy code, very flexible
Learning curve	Easier for beginners	Requires Groovy knowledge
Validation	Built-in linting/structure checks	Fewer guardrails
Recommended for	~95% of real-world use cases	Complex edge cases declarative can't express

Almost always start with Declarative — it covers nearly everything most teams need and is far easier to read/maintain.

4.3 Anatomy of a Declarative Pipeline

```
pipeline {
    agent any                       // where this runs: any available agent

    environment {
        APP_ENV = 'staging'         // environment variables available to all stages
    }

    options {
        timeout(time: 30, unit: 'MINUTES')
        timestamps()
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/you/app.git'
            }
        }

        stage('Build') {
```

```
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }

        stage('Deploy') {
            when {
                branch 'main'
            }
            steps {
                sh './deploy.sh staging'
            }
        }
    }

    post {
        success {
            echo 'Pipeline succeeded!'
        }
        failure {
            mail to: 'team@example.com', subject: 'Build failed', body: "Check ${env.BUILD
        }
    }
}
```

Here's what a typical multi-stage pipeline looks like as it runs:

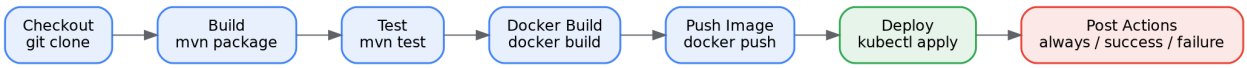

Figure 3: Typical Pipeline Stages

4.4 Key Pipeline Blocks Explained

Block	Purpose
`agent`	Where the pipeline (or a stage) executes: `any`, `none`, `label 'docker'`, or a `docker`/`kubernetes` block
`stages` / `stage`	The visible steps of your pipeline (Checkout, Build, Test, Deploy…)

Block	Purpose
`steps`	The actual commands run inside a stage
`environment`	Key-value environment variables, available as `env.VAR_NAME`
`parameters`	User-supplied inputs when triggering a build manually
`when`	Conditional logic — only run a stage if a condition is true
`post`	Actions that run after stages finish: `always`, `success`, `failure`, `unstable`, `changed`
`options`	Pipeline-wide settings: timeouts, retry, discarding old builds, etc.
`triggers`	How the pipeline automatically starts: `cron`, `pollSCM`, `upstream`
`tools`	Auto-installs/configures tools like Maven or JDK by name

4.5 Parameters — Making Builds Interactive

```
pipeline {
    agent any
    parameters {
        choice(name: 'ENVIRONMENT', choices: ['dev', 'staging', 'prod'], description: 'Whe
        string(name: 'VERSION', defaultValue: '1.0.0', description: 'App version to deploy
        booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Run test suite?'
    }
    stages {
        stage('Deploy') {
            steps {
                echo "Deploying version ${params.VERSION} to ${params.ENVIRONMENT}"
            }
        }
    }
}
```

4.6 `when` — Conditional Stage Execution

```
stage('Deploy to Prod') {
    when {
        allOf {
            branch 'main'
            expression { params.ENVIRONMENT == 'prod' }
        }
    }
    steps {
        sh './deploy.sh prod'
    }
}
```

4.7 Parallel Stages — Speeding Up Builds

```
stage('Test Suite') {
    parallel {
        stage('Unit Tests') {
            steps { sh 'mvn test' }
        }
        stage('Integration Tests') {
            steps { sh 'mvn verify -P integration' }
        }
        stage('Lint') {
            steps { sh 'npm run lint' }
        }
    }
}
```

Why: independent tasks (unit tests, linting, security scans) don't need to wait on each other — running them in parallel can cut a 15-minute pipeline down to 5 minutes.

4.8 Input Step — Manual Approval Gates

```
stage('Approve Production Deploy') {
    steps {
        input message: 'Deploy to production?', ok: 'Deploy'
    }
}
stage('Deploy to Prod') {
    steps {
        sh './deploy.sh prod'
    }
}
```

Used for **Continuous Delivery** (as opposed to full auto-deployment) — a human clicks "Deploy" in the Jenkins UI before production changes go live.

4.9 Scripted Pipeline Example (for comparison)

```
node {
    stage('Checkout') {
        git branch: 'main', url: 'https://github.com/you/app.git'
    }
    stage('Build') {
        sh 'mvn clean package'
    }
    stage('Test') {
        try {
            sh 'mvn test'
        } catch (err) {
            currentBuild.result = 'UNSTABLE'
        }
    }
}
```

Scripted pipelines give you raw Groovy — loops, custom functions, try/catch — useful when Declarative's structure can't express something unusual, but harder to read and maintain at scale.

Part 5 — Multibranch Pipelines & Shared Libraries

5.1 Multibranch Pipeline

A **Multibranch Pipeline** job automatically discovers every branch (and pull request) in a repository that contains a `Jenkinsfile`, and creates/runs a pipeline for each one — no manual job creation per branch.

Why: modern teams work with feature branches constantly; without this, you'd have to manually create a new Jenkins job every time someone opens a branch.

Setup: New Item → Multibranch Pipeline → point it at your repo (GitHub/GitLab/Bitbucket) → Jenkins scans and auto-creates a sub-job per branch found. Combine with **webhooks** so new branches/PRs are picked up instantly.

```
// A Jenkinsfile that behaves differently per branch
pipeline {
    agent any
    stages {
        stage('Build') {
            steps { sh 'mvn clean package' }
        }
        stage('Deploy to Dev') {
            when { branch 'develop' }
            steps { sh './deploy.sh dev' }
        }
        stage('Deploy to Prod') {
            when { branch 'main' }
            steps { sh './deploy.sh prod' }
        }
    }
}
```

5.2 Shared Libraries — Reusable Pipeline Code

As your Jenkinsfiles grow across many repos, you'll repeat the same logic everywhere (build steps, notification logic, deployment scripts). A **Shared Library** is a separate Git repo of reusable Groovy code that any Jenkinsfile can import — Jenkins' equivalent of a Terraform module or Ansible role.

```
(shared-library-repo)/
  vars/
    buildAndTest.groovy
  src/
    org/example/Utils.groovy
```

vars/buildAndTest.groovy:

```
def call(String buildTool = 'maven') {
    stage('Build') {
        if (buildTool == 'maven') {
            sh 'mvn clean package'
        } else {
            sh 'npm install && npm run build'
        }
    }
```

```
}
```

Using it in any project's `Jenkinsfile`:

```
@Library('my-shared-library') _

pipeline {
    agent any
    stages {
        stage('Build & Test') {
            steps {
                buildAndTest('maven')
            }
        }
    }
}
```

Register it under **Manage Jenkins → System → Global Pipeline Libraries** so every pipeline can `@Library` it by name.

Part 6 — Key Integrations

6.1 Git Integration

```
stage('Checkout') {
    steps {
        git credentialsId: 'github-creds', branch: 'main', url: 'git@github.com:you/app.gi
    }
}
```

Use **webhooks** (GitHub/GitLab → Settings → Webhooks → point at `http://your-jenkins/github-webhook/`) so pushes trigger builds instantly instead of relying on polling.

6.2 Docker Integration

Jenkins can build, run, and push Docker images directly as part of a pipeline:

```
stage('Build Docker Image') {
    steps {
        script {
            docker.build("myapp:${env.BUILD_NUMBER}")
        }
    }
}
stage('Push to Registry') {
    steps {
        script {
            docker.withRegistry('https://registry.hub.docker.com', 'dockerhub-creds') {
                docker.image("myapp:${env.BUILD_NUMBER}").push()
            }
        }
    }
}
```

You can also run the entire pipeline **inside** a Docker container as the build agent (ensures a clean, consistent environment every time):

```
pipeline {
    agent {
        docker { image 'maven:3.9-eclipse-temurin-17' }
    }
    stages {
        stage('Build') {
            steps { sh 'mvn clean package' }
        }
    }
}
```

6.3 Kubernetes Deployment

```
stage('Deploy to Kubernetes') {
    steps {
        withKubeConfig([credentialsId: 'k8s-creds']) {
```

```
            sh 'kubectl set image deployment/myapp myapp=myrepo/myapp:${BUILD_NUMBER}'
            sh 'kubectl rollout status deployment/myapp'
        }
    }
}
```

6.4 Build Tool Integration (Maven/Gradle/npm)

```
tools {
    maven 'Maven-3.9'   // auto-installed if configured under Manage Jenkins -> Tools
    jdk 'JDK-17'
}
stages {
    stage('Build') {
        steps { sh 'mvn -B clean verify' }
    }
}
```

6.5 Test Reporting

```
post {
    always {
        junit 'target/surefire-reports/*.xml'
        publishHTML(target: [reportDir: 'target/site/jacoco', reportFiles: 'index.html', r
    }
}
```

6.6 Notifications (Slack/Email)

```
post {
    failure {
        slackSend channel: '#builds', color: 'danger', message: "Build failed: ${env.JOB_N
    }
    success {
        slackSend channel: '#builds', color: 'good', message: "Build succeeded: ${env.JOB_
    }
}
```

6.7 SonarQube (Code Quality Gate)

```
stage('Code Quality') {
    steps {
        withSonarQubeEnv('MySonarServer') {
            sh 'mvn sonar:sonar'
        }
    }
}
stage('Quality Gate') {
    steps {
```

```
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}
```

Part 7 — Credentials & Security

7.1 Managing Credentials

Never hardcode passwords/API keys/SSH keys in a `Jenkinsfile`. Jenkins has a built-in, encrypted **Credentials Store**.

Manage Jenkins → Credentials → Add Credentials, choose a type:

Type	Use case
Username with password	Registry logins, basic auth APIs
SSH Username with private key	Git-over-SSH, remote server access
Secret text	API tokens, single secret strings
Secret file	Kubeconfig files, certificates
Certificate	Client TLS certs

Using a credential in a pipeline:

```
stage('Deploy') {
    steps {
        withCredentials([usernamePassword(credentialsId: 'docker-hub', usernameVariable: '
            sh 'docker login -u $USER -p $PASS'
        }
    }
}
```

Why this matters: credentials are encrypted at rest, masked in build logs automatically (shown as ****), and access can be scoped per-folder/per-job so not every pipeline can use every secret.

7.2 Role-Based Access Control (RBAC)

By default, a fresh Jenkins install has one admin. For real teams, install the **Role-based Authorization Strategy plugin** to define fine-grained permissions:

- **Global roles** — e.g., “read-only viewer,” “admin.”
- **Project/folder roles** — e.g., “team-A can only build/configure jobs inside the `team-a/` folder.”

Why: without RBAC, any user with Jenkins access could modify or delete any job, view any credential usage, or trigger production deployments — a serious risk in multi-team environments.

7.3 Securing Jenkins — Checklist

- Enable **CSRF protection** (on by default in modern Jenkins).
- Run Jenkins **behind HTTPS** (reverse proxy with Nginx/Apache + TLS cert, or a load balancer).
- Disable the built-in **Groovy script console** access for non-admins (it allows arbitrary code execution).
- Keep Jenkins **core and plugins updated** — CI servers are a common attack target since a breach can compromise your entire deployment pipeline.
- Restrict **agent-to-controller** access (Agent → Controller Security) so a compromised agent can’t take over the controller.

- Use the **Credentials Binding** plugin, never plaintext secrets in `Jenkinsfile` or job configs.
- Audit with the **Audit Trail plugin** to log who changed what.

7.4 Backup & Disaster Recovery

```
# Simple backup: archive the whole JENKINS_HOME
tar -czf jenkins_backup_$(date +%F).tar.gz /var/lib/jenkins

# Restore: stop Jenkins, extract into JENKINS_HOME, restart
sudo systemctl stop jenkins
tar -xzf jenkins_backup_2026-07-14.tar.gz -C /
sudo systemctl start jenkins
```

For production, use the **ThinBackup plugin** (scheduled, incremental) or back up the underlying volume/disk on a schedule.

Part 8 — Plugins & Blue Ocean

8.1 The Plugin Ecosystem

Plugins extend nearly every part of Jenkins: SCM integrations, build tools, cloud providers, notification channels, UI themes, security. **Manage Jenkins → Plugins** to browse, install, and update.

Essential plugins for most teams:

Plugin	Purpose
Git / GitHub	Source control integration
Pipeline	Enables Jenkinsfile-based pipelines
Credentials Binding	Secure secret injection
Docker Pipeline	Build/push/run Docker images
Kubernetes	Dynamic pod-based build agents
Blue Ocean	Modern visual pipeline UI
Slack Notification	Build alerts
JUnit	Test result reporting
Role-based Authorization Strategy	RBAC
SonarQube Scanner	Code quality gates

Why be selective: every plugin is a potential attack surface and maintenance burden — install what you need, keep the list lean, and update regularly.

8.2 Blue Ocean — Modern Pipeline Visualization

Blue Ocean is a plugin (and UI) that renders pipelines as a clear, interactive visual flow — showing exactly which stage passed/failed/is running, with inline logs, instead of the older, denser classic UI.

Access it at `http://your-jenkins/blue` after installing the plugin. It's purely a visualization/editing layer on top of the same underlying Pipeline engine — it doesn't change how Jenkinsfiles work.

Part 9 — Full Practical Project: End-to-End CI/CD Pipeline

Goal: on every push to `main`, build a Java app, run tests, build a Docker image, push it to a registry, and deploy it to Kubernetes — with Slack notifications on failure.

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "myrepo/myapp"
        IMAGE_TAG  = "${env.BUILD_NUMBER}"
    }

    options {
        timeout(time: 45, unit: 'MINUTES')
        disableConcurrentBuilds()
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', credentialsId: 'github-creds', url: 'git@github.com:yo
            }
        }

        stage('Build') {
            steps {
                sh 'mvn -B clean package -DskipTests'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }

        stage('Code Quality') {
            steps {
                withSonarQubeEnv('MySonarServer') {
                    sh 'mvn sonar:sonar'
                }
            }
        }

        stage('Build Docker Image') {
            steps {
                script {
                    dockerImage = docker.build("${IMAGE_NAME}:${IMAGE_TAG}")
```

```
                }
            }
        }

        stage('Push Image') {
            steps {
                script {
                    docker.withRegistry('https://index.docker.io/v1/', 'dockerhub-creds')
                        dockerImage.push()
                        dockerImage.push('latest')
                    }
                }
            }
        }

        stage('Approve Production Deploy') {
            when { branch 'main' }
            steps {
                input message: "Deploy ${IMAGE_TAG} to production?", ok: 'Deploy'
            }
        }

        stage('Deploy to Kubernetes') {
            when { branch 'main' }
            steps {
                withKubeConfig([credentialsId: 'k8s-creds']) {
                    sh "kubectl set image deployment/myapp myapp=${IMAGE_NAME}:${IMAGE_TAG
                    sh 'kubectl rollout status deployment/myapp --timeout=120s'
                }
            }
        }
    }

    post {
        success {
            slackSend channel: '#builds', color: 'good',
                message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} deployed ${IMAGE_T
        }
        failure {
            slackSend channel: '#builds', color: 'danger',
                message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} — check ${env.BUILD
        }
        always {
            cleanWs()   // clean the workspace after every run
        }
    }
}
```

Walking through why each piece is there: - `disableConcurrentBuilds()` prevents two deploys racing each other. - Tests run **before** the Docker image is built — no point packaging broken code. - The **SonarQube quality gate** stops the pipeline if code quality drops below the team's bar. - The **`input` step** turns this into Continuous *Delivery* rather than fully automatic deployment — a human confirms production releases. - `post { always { cleanWs() } }` keeps the agent's disk from filling up with old build artifacts over time.

Part 10 — Best Practices & Troubleshooting

10.1 Best Practices

- **Always use Pipeline-as-Code (`Jenkinsfile`)**, committed to the same repo as the application — never rely on UI-only Freestyle jobs for anything beyond simple experiments.
- **Keep the controller lightweight** — run actual builds on agents, not the controller itself.
- **Use dynamic agents (Docker/Kubernetes)** where possible — every build starts in a clean, disposable environment, eliminating "works on my agent" drift.
- **Fail fast** — put quick checks (lint, unit tests) before slow ones (integration tests, deployment).
- **Parallelize independent stages** to cut pipeline duration.
- **Pin tool/plugin versions** to avoid unexpected breakage after updates.
- **Store secrets only in the Credentials store**, never in `Jenkinsfile` or environment variables checked into Git.
- **Archive build artifacts and test reports** so failures are diagnosable after the fact.
- **Use Shared Libraries** once you have more than a couple of similar Jenkinsfiles across repos.
- **Clean up workspaces** and old builds (`options { buildDiscarder(logRotator(numToKeepStr: '20')) }`) to avoid disk exhaustion.

10.2 Common Beginner Mistakes

- Running heavy builds directly on the controller instead of agents (can crash the entire Jenkins instance under load).
- Forgetting `when { branch 'main' }` guards, so a Pull Request pipeline accidentally deploys to production.
- Hardcoding secrets in shell steps (`sh "curl -u admin:password123 ..."`) instead of `withCredentials`.
- Not archiving test reports, making failures hard to diagnose after the workspace is cleaned.
- Letting old builds/artifacts pile up indefinitely, filling agent disks.
- Using `agent any` for every stage instead of scoping agents per-stage when different environments are needed.

10.3 Troubleshooting Commands & Tips

```
# Tail the Jenkins system log (native install)
sudo journalctl -u jenkins -f

# Check Jenkins is listening
curl -I http://localhost:8080

# Groovy console (Manage Jenkins -> Script Console) — inspect a job programmatically
Jenkins.instance.getItem('my-job').builds.each { println it.number }

# Restart Jenkins safely (waits for running builds to finish)
http://your-jenkins/safeRestart

# Common fix: "Permission denied" on a build step
# -> check the agent's OS user has correct file/docker-socket permissions

# Common fix: Pipeline stuck "waiting for next available executor"
```

```
# -> no agent matches the required label, or all executors are busy — add
#    agents or increase executor count (Manage Jenkins -> Nodes -> configure)
```

10.4 Reading a Failed Pipeline

1. Open the build → **Console Output** — read from the bottom up first (the actual error is usually near the end).
2. Check **which stage** failed in the Stage View / Blue Ocean visualization.
3. Check `post { failure { ... } }` — did notifications fire, and is the reported reason accurate?
4. If it's an agent/environment problem, SSH into the agent directly and try to reproduce the failing step manually.

Appendix A — Pipeline Syntax Cheat Sheet

```
pipeline {
    agent any / none / { label 'linux' } / { docker { image '...' } }

    environment { VAR = 'value' }
    parameters { string(...) / choice(...) / booleanParam(...) }
    options { timeout(...) / retry(...) / disableConcurrentBuilds() / buildDiscarder(...)
    triggers { cron('H 2 * * *') / pollSCM('H/5 * * * *') }
    tools { maven 'M3' / jdk 'JDK17' }

    stages {
        stage('Name') {
            when { branch 'main' / expression { } / allOf { } / anyOf { } }
            steps {
                sh 'command'                  // Linux shell
                bat 'command'                  // Windows batch
                echo 'message'
                git url: '...', branch: '...'
                script { /* arbitrary groovy */ }
            }
            post { always {} success {} failure {} }
        }
    }

    post {
        always {}
        success {}
        failure {}
        unstable {}
        changed {}
    }
}
```

Appendix B — Useful Jenkins CLI Commands

```
# Download the CLI jar from your Jenkins instance
curl -O http://your-jenkins/jnlpJars/jenkins-cli.jar

java -jar jenkins-cli.jar -s http://your-jenkins/ list-jobs
java -jar jenkins-cli.jar -s http://your-jenkins/ build my-job
java -jar jenkins-cli.jar -s http://your-jenkins/ console my-job
java -jar jenkins-cli.jar -s http://your-jenkins/ install-plugin git
java -jar jenkins-cli.jar -s http://your-jenkins/ safe-restart
```

Appendix C — Quick Interview / Revision Q&A

Q: What's the difference between Continuous Delivery and Continuous Deployment?
A: Both automatically build, test, and package every change. Continuous Delivery stops at a manual approval gate before production; Continuous Deployment pushes straight to produc-

tion with no human intervention.

Q: Why is Pipeline-as-Code (Jenkinsfile) preferred over Freestyle jobs? A: It's versioned alongside application code, reviewable via pull requests, portable across Jenkins instances, and supports complex logic (parallelism, conditionals) that Freestyle's UI-based config can't express well.

Q: What is the difference between the Jenkins controller and an agent? A: The controller manages scheduling, configuration, and the UI; agents are the machines that actually execute build steps. Separating them avoids overloading the controller and allows builds to run on the right OS/toolset.

Q: How do you keep secrets out of a Jenkinsfile? A: Store them in the Jenkins Credentials store and inject them at runtime with `withCredentials`, which also masks them in build logs automatically.

Q: What's a Multibranch Pipeline and why use one? A: A job type that auto-discovers every branch/PR containing a Jenkinsfile and runs a pipeline for each — eliminating manual job creation per branch in active, multi-branch repositories.

Q: When would you use a Shared Library? A: When multiple repositories/Jenkinsfiles repeat the same logic (build steps, notification patterns, deployment routines) — a Shared Library centralizes that code so it's written and maintained once.

Q: How does Jenkins achieve dynamic, disposable build environments? A: Via the Docker or Kubernetes plugin, which spins up a fresh container/pod as the build agent for each run and tears it down afterward, avoiding environment drift between builds.

Closing Notes

You now have a complete path from zero to a production-style CI/CD pipeline:

- Install Jenkins locally (Docker is the fastest way to experiment).
- Recreate the Freestyle job example, then rebuild the same thing as a Declarative Pipeline to feel the difference directly.
- Add Git webhooks so builds trigger automatically instead of manually.
- Build the full end-to-end pipeline in Part 9 against your own small app — even a “hello world” service is enough to practice every stage: build, test, containerize, push, approve, deploy.
- Once comfortable, add a Shared Library and Multibranch Pipeline as your projects multiply, and lock down credentials/RBAC before anything touches real production systems.

Practice by intentionally breaking a pipeline (bad test, missing credential, wrong branch condition) and using the Console Output to diagnose it — that debugging loop is what actually builds Jenkins fluency.